

RANSOMWARE CTF ANALYSIS & DECRYPTION

FIELD REFERENCE & CRYPTOGRAPHIC EXPLOITATION MANUAL

1. RANSOMWARE ANALYSIS METHODOLOGY

1. Triage & Reconnaissance

- **Identify Locked Files:** Note file extensions (e.g., `.enc`, `.locked`), modified timestamps, and target directory paths.
- **Locate Artifacts:** Search the directory for ransom notes (`README.txt`, GUI executables), public keys (`public.pem`), or leftover build/decryption scripts (`encryptor.py`).
- **Identify Execution Context:** Check running processes or command history (`history`, `ps aux`) to determine how the ransomware was executed.

2. Encryption Mechanism Analysis

Most ransomware implementations utilize a multi-layered hybrid encryption architecture:

- **Symmetric Encryption (AES / ChaCha20):** Used to encrypt large payload files rapidly.
- **Asymmetric Encryption (RSA / ECC):** Encrypts the symmetric key using a Public Key so only the attacker (holding the Private Key) can unlock it.
- **Reverse Engineering (Binaries):** Use tools like Ghidra, IDA Free, or dnSpy (.NET) to locate hardcoded keys, custom XOR logic, or weak PRNG functions.
- **Script Inspection (Python / Bash):** Review source code directly to extract key parameters, initialization vectors (IVs), or mathematical constants.

3. Exploiting Cryptographic Flaws

- **Weak PRNG (Pseudo-Random Number Generator):** Keys generated using `time()` or low-entropy seeds can be brute-forced or recalculated by matching the timestamp.
- **RSA Implementation Vulnerabilities:**
 - *Small Modulus (N):* Easily factorized into prime factors p and q .
 - *Small Public Exponent ($e = 3$):* Vulnerable to Hastad's Broadcast Attack or direct root extraction.
 - *Close Primes:* Vulnerable to Fermat's Factorization.
 - *Small Private Exponent (d):* Vulnerable to Wiener's or Boneh-Durfee attacks.
- **Reused Key Streams:** If stream ciphers or XOR are used without unique IVs/nonces, XORing two ciphertext files together cancels out the key stream ($C_1 \oplus C_2 = P_1 \oplus P_2$).

2. ESSENTIAL TOOL SET

CATEGORY	TOOL	PRIMARY USE CASE
RSA Factorization	RsaCtfTool	Automated attack suite for exploiting weak RSA keys and recovering private exponent d .
Prime Lookup	Factordb	

CATEGORY	TOOL	PRIMARY USE CASE
		Online database and API to instantly retrieve known prime factors of modulus N .
Static Analysis	Ghidra / IDA Pro	Decompiling binary executables to trace encryption routines, hardcoded logic, and key handling.
Memory Forensics	Volatility 3	Extracting unencrypted symmetric keys, nonces, or plaintexts directly from RAM dumps.
Crypto Libraries	PyCryptodome	Writing custom Python scripts to rebuild keys, construct structures, and handle AES/RSA decryption.
Key Inspection	OpenSSL	Parsing, converting, manipulating, and utilizing asymmetric/symmetric cryptographic keys via CLI.

3. COMPREHENSIVE OPENSSL REFERENCE

OpenSSL is an open-source command-line tool and library implementing SSL/TLS protocols alongside a robust cryptographic engine for encryption, signing, and key generation.

KEY PARAMETER DEFINITIONS

- N (Modulus): The product of two large prime numbers ($N = p \times q$). Public.
- e (Public Exponent): The value used to encrypt data (commonly 65537). Public.
- d (Private Exponent): The modular multiplicative inverse of $e \pmod{\phi(N)}$. Kept secret.
- p, q : Prime factors of N . Kept secret.

Inspect Public Key Details (Extract N and e):

```
openssl rsa -pubin -in public.pem -text -noout
```

Inspect Private Key Details (Extract p, q, d):

```
openssl rsa -in private.pem -text -noout
```

Decrypt a File Directly with an RSA Private Key:

```
openssl pkeyutl -decrypt -inkey private.pem -in file.docx.enc -out file.docx
```

Decrypt a File using AES-256-CBC:

```
openssl enc -d -aes-256-cbc -in file.enc -out file.txt -k "SecretPassword" -pbkdf2
```

Convert Key Formats (DER to PEM):

```
openssl rsa -inform DER -in key.der -outform PEM -out key.pem
```

4. RECONSTRUCTING RSA PRIVATE KEYS IN PYTHON

If you factorize N into p and q using tools like Factordb, use the following Python workflow with PyCryptodome to reconstruct the private key and decrypt your target files:

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP

# 1. Defined parameters extracted from analysis
p = 123456789... # Prime factor 1
q = 987654321... # Prime factor 2
e = 65537

# 2. Compute RSA variables
n = p * q
phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

# 3. Construct RSA Private Key Structure
key = RSA.construct((n, e, d, p, q))

# 4. Export Key to PEM format
with open("recovered_private.pem", "wb") as f:
    f.write(key.export_key("PEM"))

# 5. Decrypt Encrypted Payload
with open("school_report.docx.enc", "rb") as f:
    ciphertext = f.read()

# Using raw RSA or OAEP depending on implementation
cipher = PKCS1_OAEP.new(key)
plaintext = cipher.decrypt(ciphertext)

with open("school_report.docx", "wb") as f:
    f.write(plaintext)
```

5. CRITICAL OPERATIONAL TIPS FOR CTFs

★ CTF EXECUTION DIRECTIVES

- **Check the Source Files:** Always read local directory scripts first. If the encryption script used `random.seed(timestamp)`, calculate the epoch time of the file creation date to instantly regenerate the exact same key.
- **Inspect the Ransom Note:** Ransom notes often contain hidden base64 strings, metadata, or fake bitcoin wallet strings designed to act as hints or flags.
- **Keep Backup Files:** Always copy encrypted files to a `/tmp` directory before testing decryption scripts to avoid corrupting the original challenge artifacts.